



TRƯỜNG ĐẠI HỌC KINH TẾ - KỸ THUẬT CÔNG NGHIỆP
KHOA: KHOA HỌC ỨNG DỤNG.

CHƯƠNG 2

PHÂN TÍCH TẬP TIN XML, JSON

Giảng viên: TS.Cao Diệp Thắng
Khoa: Khoa học Ứng dụng

Chương 2.

MỤC TIÊU

- + Cách định dạng và cấu trúc dữ liệu của tập tin XML và JSON.
- + Các thư viện tích hợp sẵn để phân tích dữ liệu XML và JSON.
- + Cách xử lý các trường hợp lỗi có thể xảy ra, như thiếu thuộc tính, phần tử hoặc cú pháp không đúng.
- + Cách gửi và nhận dữ liệu qua API: Khi làm việc với tập tin XML và JSON từ nguồn trực tuyến

NỘI DUNG BÀI HỌC

2.1

TÀI LIỆU XML

2.2

PYTHON JSON

MỤC TIÊU BÀI HỌC

Sau khi học xong bài này, người học sẽ nắm được các vấn đề sau:

- Cách định dạng và cấu trúc dữ liệu của tập tin XML và JSON.
- Các thư viện tích hợp sẵn để phân tích dữ liệu XML và JSON.
- Cách xử lý các trường hợp lỗi có thể xảy ra, như thiếu thuộc tính, phần tử hoặc cú pháp không đúng.
- Cách gửi và nhận dữ liệu qua API: Khi làm việc với tập tin XML và JSON từ nguồn trực tuyến

2.1. TÀI LIỆU XML

2.1.1. Khái niệm tài liệu XML

a. Khái niệm: XML (**Ex**tensible **M**arkup **L**anguage) là ngôn ngữ đánh dấu mở rộng.

- ❑ XML được tổ hợp web toàn cầu (**W**orld **W**ide **W**eb **C**onsortium) W3C đề nghị tạo ra với mục đích phát triển và mở rộng các ngôn ngữ đánh dấu khác ngoài định dạng .txt, ASCII,...
- ❑ Cách lưu trữ file XML này khá đơn giản, nhưng nó có thể mô tả được nhiều dữ liệu khác nhau. XML có nhiệm vụ truyền dữ liệu và mô tả những loại dữ liệu khác nhau.
- ❑ XML được sử dụng với mục đích lưu trữ, cấu trúc khi trao đổi dữ liệu giữa các ứng dụng và để lưu trữ dữ liệu. Vì vậy việc xây dựng file XML tương đối hữu ích trong việc chia sẻ dữ liệu giữa các hệ thống, chương trình.[2]

Ghi nhớ:

- XML là ngôn ngữ đánh dấu được dùng để miêu tả dữ liệu.
- Các thẻ tag trong file XML chưa được xác định trước. Do đó, người dùng phải tự định nghĩa trong quá trình tạo file XML.
- Quá trình thao tác cũng như truyền dữ liệu xuất hiện tỉ lệ sai sót lên đến 5 - 7%

b. Mục đích của file .xml

Chia sẻ tài nguyên dữ liệu giữa các platform trên các hệ thống khác nhau

Ví dụ: Một hệ thống quản lý nhà hàng muốn lấy thông tin khách du lịch từ hệ thống công ty lữ hành thì giữa hệ thống cần đảm bảo thực hiện các thao tác công việc sau:

- Các hệ thống cần có sự thống nhất về cấu trúc file XML.
- Công ty lữ hành cần có trích xuất dữ liệu từ hệ thống của mình, đóng gói dữ liệu dưới dạng file XML theo cấu trúc đã thỏa thuận trước đó.

c. Đặc điểm của XML

1. XML là ngôn ngữ có tính chất đánh dấu.
2. Được thiết kế để lưu trữ và truyền tải dữ liệu, xuất hiện theo chuẩn W3C từ 1998 và có sẵn như một chuẩn mở.
3. XML chỉ áp dụng cho những kiểu dữ liệu có cấu trúc cụ thể.
4. Về mặt đánh giá trực quan, XML có cấu trúc tương tự với file HTML.
5. XML một dạng file văn bản nhưng chỉ mang dữ liệu chứ không hiển thị nó (không dùng để đọc).
6. Cấu trúc file XML thường rất dài, có thể tự mô tả dữ liệu.
7. File XML được biết đến là một dạng module.
8. XML được ví như một nền tảng cho web ngữ nghĩa và RDF.
9. XML được coi như một file cầu nối giữa hệ thống HTML và XHTML.
10. File XML được miễn phí bản quyền, có platform độc lập và được hỗ trợ rất tốt cho người dùng.

d. Ưu và nhược điểm của File XML

Ưu điểm

- Tính độc lập rất cao, đây là ưu điểm lớn nhất của file XML, chúng được sử dụng phổ biến dưới phần mô tả ở dạng text.
- Dễ dàng đọc và phân tích các nguồn dữ liệu.
- File XML được tạo một cách đơn giản.

Nhược điểm

Tỷ lệ sai sót trong khi sử dụng định dạng file XML lên tới 5 – 7% → phải cân nhắc kỹ lưỡng trước khi lựa chọn XML để trao đổi thông tin

Lưu ý: đa phần các lỗi khi sử dụng file XML đều xuất phát từ người dùng cấu hình sai

e. Cú pháp tài liệu XML

Thẻ mở Thẻ đóng

`<nodename>nội dung</nodename>`

Trong đó:

- `<nodename>` là thẻ mở, tên của thẻ này do người dùng tự định nghĩa.
- `</nodename>` là thẻ đóng, tên của thẻ này **phải trùng** với tên của thẻ mở.
- content là nội dung của thẻ này

Ví dụ

`<domain>uneti.edu.vn</domain>`

Để bổ sung các thuộc tính vào các thẻ XML, sử dụng cú pháp:

`<nodename ten_thuoc_tinh="giá trị">content</nodename>`

Ví dụ

Thuộc tính Nội dung thẻ company

`<company director="Nguyễn Văn A"email="van.A@gmail.com">Dat Viet</company>`

Gía trị thuộc tính

❑ Khai báo Header (Chỉ thị xử lý)

Cú pháp `<?xml version="1.0" encoding="UTF-8"?>`

- Version: thông tin về version XML đang sử dụng
 thuộc tính version là bắt buộc; thiếu hoặc viết sai cú pháp sẽ khiến file XML không hợp lệ (not well-formed).
- encoding (kiểu mã hóa ký tự) : **UTF-8**, **UTF-16**, ISO-10646-UCS-2, ISO-10646-UCS-4, ISO-8859-1 to ISO-8859-9, ISO-2022-JP, Shift_JIS, EUC-JP

❑ Root node:

Thẻ ngoài cùng gọi là **root node**. Thẻ này sẽ khai báo tên chính của tài liệu XML.

Ví dụ 2.1.1.1.xml

1	<code><?xml version="1.0" encoding="UTF-8"?></code>
2	<code><lists></code>  Root Node
3	<code><company director="Nguyễn Văn A" email="van.A@gmail.com">DatViet</company></code>
4	<code><company director="Trần Văn C" email="van.C@gmail.com">Hoàng Mai</company></code>
5	<code><company director="Lê Thi B" email="thi.B@gmail.com">Viet Hung</company></code>
6	<code></lists></code>

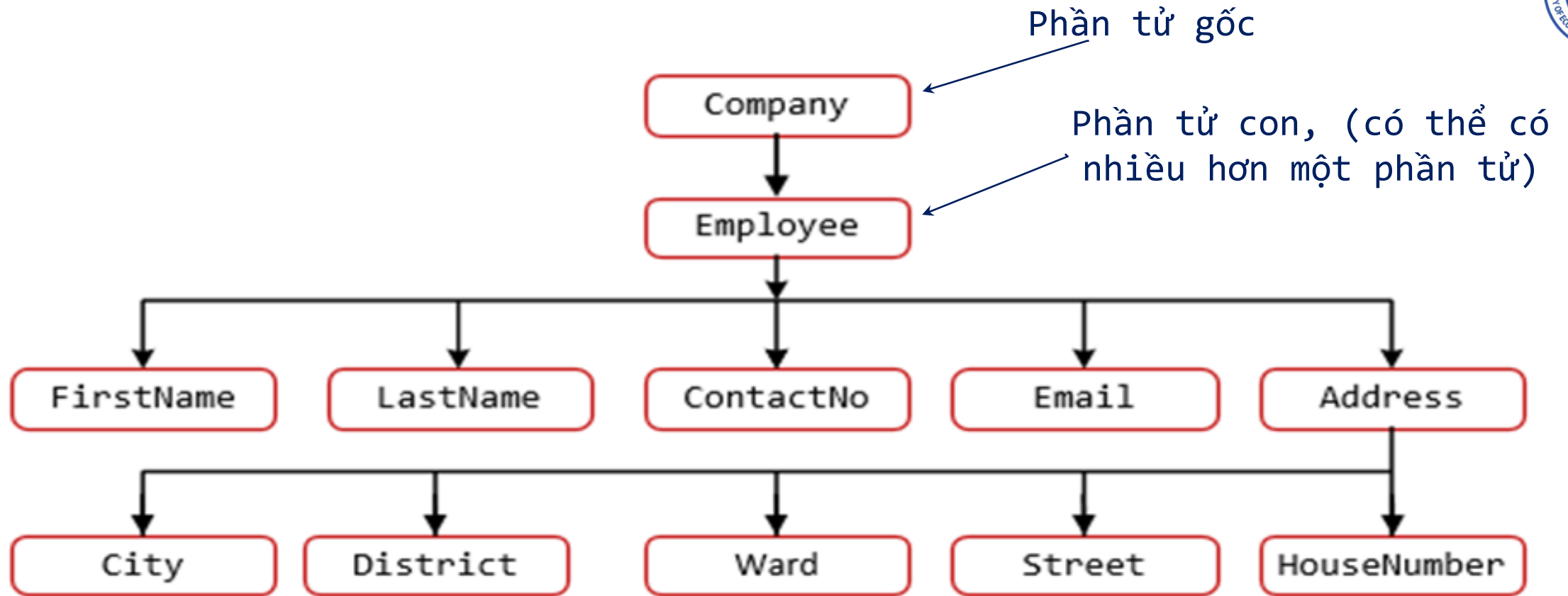
g. Cấu trúc cây (Tree Structure) trong XML

- Cấu trúc cây chứa các phần tử gốc (**root node**) hay phần tử cha, các phần tử con, ...

Ví dụ 2.1.1.2.xml

1	<?xml version="1.0"?>	
2	<Company>	
3	<Employee>	
4	<FirstName>Nguyễn</FirstName>	
5	<LastName>Văn A</LastName>	
6	<ContactNo>0901234567</ContactNo>	
7	<Email>van.a@uneti.edu.vn</Email>	
8	<Address>	
9	<City>Hà Nội</City>	
10	<District>Hai Bà Trưng</District>	
11	<Ward>Vĩnh Tuy</Ward>	
12	<Street>Minh Khai</Street>	
13	<HouseNumber>456</HouseNumber>	
14	</Address>	
15	</Employee>	
16	</Company>	

Thông tin nhân viên của công ty được lưu trữ trong một phần tử **<Employee>** các thuộc tính của nhân viên **tên, họ, điện thoại, email, địa chỉ** và các thông tin liên quan đến công việc của họ được lưu trữ trong các phần tử con.



Hình 2.1. Sơ đồ cây của file employee.xml

h. Element trong XML

Phần tử/thẻ XML

- Các thẻ XML bao gồm tất cả mọi thứ từ thẻ khai báo trên **header** cho đến thẻ **Root**, các thẻ con, ... cho đến thẻ cuối cùng trong tài liệu.
- Một thẻ XML dùng để chứa dữ liệu dạng đơn giản (number, string) cho đến phức tạp (thẻ XML chứa thẻ XML).

Ví dụ 2.1.1.3. Viết một file XML để lưu trữ danh sách sinh viên gồm các thông tin:

- Tên sinh viên (tensv)
- Năm sinh (namsinh)
- Giới tính (gioitinh)

Cấu trúc cây XML lưu trữ danh sách sinh viên

Ví dụ 2.1.1.3.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <ds-sinhvien>
3     <sinhvien>
4         <tensv>Nguyễn Văn A</tensv>
5         <namsinh>2004</namsinh>
6         <gioitinh>Nam</gioitinh>
7     </sinhvien>
8 </ds-sinhvien>
```

Thẻ XML rỗng và rút gọn thẻ đóng

Trường hợp thẻ sinh viên không có dữ liệu → để trống,

1	<?xml version="1.0" encoding="UTF-8"?>
2	<ds-sinhvien>
3	<sinhvien></sinhvien>
4	</ds-sinhvien>

Trường hợp thẻ sinh viên không có dữ liệu → viết rút gọn thẻ đóng.

1	<?xml version="1.0" encoding="UTF-8"?>
2	<ds-sinhvien>
3	<sinhvien/>
4	</ds-sinhvien>

Quy tắc liên quan đến XML Element

Đặt tên thẻ

- Tên thẻ phải bắt đầu là một chữ cái hoặc ký tự gạch dưới.
- Tên thẻ không được bắt đầu bằng XML (hoặc Xml, xMl, xmL) vì nó giống với thẻ khai báo header.
- Tên thẻ không được chứa khoảng trắng.
- Tên thẻ có thể chứa chữ cái, chữ số, dấu gạch dưới, dấu gạch ngang, dấu chấm và dấu hai chấm.

Cấu trúc thẻ:

- Mỗi thẻ mở đều phải có thẻ đóng và chúng phải tuân theo đúng thứ tự.
- Có thể dùng thẻ đóng rút gọn nếu như thẻ đó rỗng.

Naming styles:

Bảng 2.1. Minh họa một số cách đặt tên thẻ XML

Style	Example	Description
Lower case	<firstname>	Tất cả đều chữ in thường
Upper case	<FIRSTNAME>	Tất cả đều chữ in hoa
Underscore	<first_name>	Các từ cách nhau bởi dấu gạch dưới
Pascal case	<FirstName>	Viết hoa ký tự đầu tiên trong các từ
Camel case	<firstName>	Viết hoa ký tự đầu tiên trong từ thứ hai trở đi.

Lưu ý: nên đặt tên thẻ là một danh từ, nên đặt tên bằng tiếng Anh vì nó đơn giản và phổ dụng trên thế giới.

i. Thuộc tính XML (XML Attributes)

Thuộc tính XML là những dữ liệu được khai báo bên trong thẻ mở theo cú pháp sau:

1	<element_name property_name="property_value">
2	
3	</element_name>

Mỗi thẻ XML (XML element) có thể không có thuộc tính nào hoặc có nhiều thuộc tính tùy vào cách định nghĩa của lập trình viên.

Ví dụ 2.1.1.4. Tập tin **library.xml**

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <library>
3   <book id="1">
4     <title>The Great Gatsby</title>
5     <author>F. Scott Fitzgerald</author>
6     <genre>Fiction</genre>
7     <price currency="USD">10.99</price>
8   </book>
9   <book id="2">
10    <title>To Kill a Mockingbird</title>
11    <author>Harper Lee</author>
12    <genre>Fiction</genre>
13    <price currency="USD">12.99</price>
14  </book>
15 </library>
```

i. XML Namespace

Ví dụ 2.1.1.5: Bị trùng lặp thẻ.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <ShopOrder>
3     <Order>
4         <Customer>
5             <Title>Nguyễn Văn A</Title>
6             <Address>Đồng Văn - Lạng Sơn</Address>
7         </Customer>
8         <Product>
9             <Title>Áo khoác</Title>
10            <Quantity>20</Quantity>
11            <Price>500.000 VNĐ</Price>
12        </Product>
13    </Order>
14 </ShopOrder>
```

Thẻ tên khách hàng

Thẻ tên sản phẩm

Trùng thẻ

Trong tài liệu XML trên có hai thẻ Title → xung đột thẻ XML → cần sử dụng XML Namespace

Cú pháp tạo XML namespace:

Để tạo một namespace thì ta sẽ khai báo một thuộc tính **xmlns:url**.

Trong đó

- **xmlns** là viết tắt của từ XML namespace.
- **url** (Uniform Resource Locator) là đường dẫn URL của namespace, có thể là một địa chỉ nào đó trên internet hoặc một địa chỉ nào đó đều được nhưng phải đảm bảo rằng nó là duy nhất trong file tài liệu XML của người dùng.

Ví dụ 2.1.1.5.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <ShopOrder>
3   <Order>
4     <cus:Customer
5       xmlns:cus="https://Datviet.com.vn/customer">
6       <cus:Title>Nguyễn Văn A</cus:Title>
7       <cus:Address>Đồng Văn – Lạng Sơn</cus:Address>
8     </cus:Customer>
9   </Order>
10 </ShopOrder>
```

các thẻ XML phải bổ sung tên của namespace theo cú pháp namespace_name:element_name.



k. Ứng dụng file XML trong thực tiễn

XML thường được dùng làm cơ sở cho các định dạng tài liệu khác nhau.

- + ATOM và RSS: mô tả cách ứng dụng đọc xử lý nguồn cấp web.
- + Microsoft Office từ phiên bản 2007 và các phiên bản tiếp theo đều sử dụng XML làm cơ sở cấu trúc tài liệu. Một số định dạng tài liệu chúng ta có thể thấy như **.docx**, chữ "X" được sử dụng xuất phát từ file XML. Bên cạnh đó, người dùng cũng có thể gặp định dạng này trong excel và power point.
- + Microsoft .NET cũng sử dụng XML cho các file có cấu hình nhỏ.

Đọc file XML

- + Công cụ chỉnh sửa và biên tập file XML: Altova XMLSpy Enterprise Edition, XML Notepad, Notepad++,...
- + Công cụ chuyển đổi định dạng XML: Word Cleaner, PDF to XML, Total CSV Converter, AZ XML to PDF Converter,...

Có thể xem nội dung file XML bằng một trong các trình duyệt:

Firefox/Chrome/Internet Explorer/Edge/Opera, ...

2.1.2. Phân tích tài liệu XML



Có thể sử dụng các module như:

minidom, ElementTreexml,

cả hai đều cung cấp phương pháp phân tích DOM (***D**ocument **O**bject **M**odel*) cho tài liệu XML.

Mỗi cách có những ưu điểm và hạn chế riêng, do đó, việc lựa chọn cách phù hợp sẽ phụ thuộc vào yêu cầu cụ thể của bài toán thực tế.

a. Khái niệm DOM.



□ DOM là tên viết tắt của (*Document Object Model* – có thể hiểu là Mô hình đối tượng tài liệu) là một chuẩn giao diện lập trình (API) *độc lập nền tảng* và *độc lập ngôn ngữ*, cho phép các chương trình, và các ngôn ngữ kịch bản (vd: JavaScript, Python, PHP.v.v.) **truy cập và cập nhật động** nội dung, cấu trúc và của tài liệu HTML/XML.

Cách DOM làm việc:

- Biểu diễn toàn bộ tài liệu dưới dạng **cấu trúc cây phân cấp (tree structure)** của các **Nút (Nodes): Document, Element, Text, Attr, comment, ...**
- Qui chiếu mỗi nút thành một đối tượng (Objects) có các thuộc tính (dữ liệu) và phương thức (hành vi), cho phép thực hiện các thao tác (*duyệt, tìm kiếm, và chỉnh sửa*) một cách hệ thống.

b. XML DOM



XML DOM là một API (**A**pplication **P**rogramming **I**nterface) tiêu chuẩn cho việc truy xuất và thao tác dữ liệu trong tài liệu XML bằng cách sử dụng DOM. Nó cho phép dễ dàng truy xuất các phần tử, thuộc tính và nội dung của tài liệu XML bằng cách sử dụng các phương thức của API.

□ Cấu trúc cây:

- Một tài liệu XML được phân tích thành một cây gồm các nút.
- Nút gốc (**root node**) là phần tử chính của tài liệu XML.
- Các nút con (**child nodes**) bao gồm các phần tử, văn bản, hoặc thuộc tính bên trong phần tử cha.
- Mỗi nút có thể có cha, con, hoặc anh em (siblings).

❑ Các loại nút trong DOM:

Theo XML DOM, mọi thứ trong tài liệu XML đều là một nút (node):

- ❖ **Document Node**: (hay **#document**) là nút gốc cao nhất trong cây DOM, đại diện cho toàn bộ tài liệu XML. Nó chứa tất cả các thành phần của tài liệu, bao gồm:
 - **Processing Instruction Node** (mã số 7): Chỉ thị xử lý (như `<?xml-stylesheet ... ?>`)
 - **Comment Nodes** (mã số 8) : các bình luận/chú thích như (`<!-- This is a comment -->`)
 - **Element Node** (mã số 1) : mỗi phần tử thẻ XML là một nút (ví dụ `<book>`, `<title>`)
 - **Text Nodes** (mã số 3): Văn bản trong các phần tử XML là các nút văn bản (vd: "Ng Văn A" trong `<title> Ng Văn A <title>`)
 - **Attribute Node** (mã số 2): đại diện cho các thuộc tính của thẻ(vd: `id="1"` trong `<book id="1">`)
 - **DocumentType Node** (mã số 10): Đại diện cho khai báo DOCTYPE (ví dụ `<!DOCTYPE bookstore>`), nếu có trong tài liệu XML. Nó là nút con trực tiếp của Document Node

Ví dụ 2.1.2.1. Xét cấu trúc file 'library_.xml' sau:



Document node (mã số 9) :node gốc của toàn bộ tài liệu, chứa tất cả các node con.

Nút gốc trong XML được đặt tên là <library>.

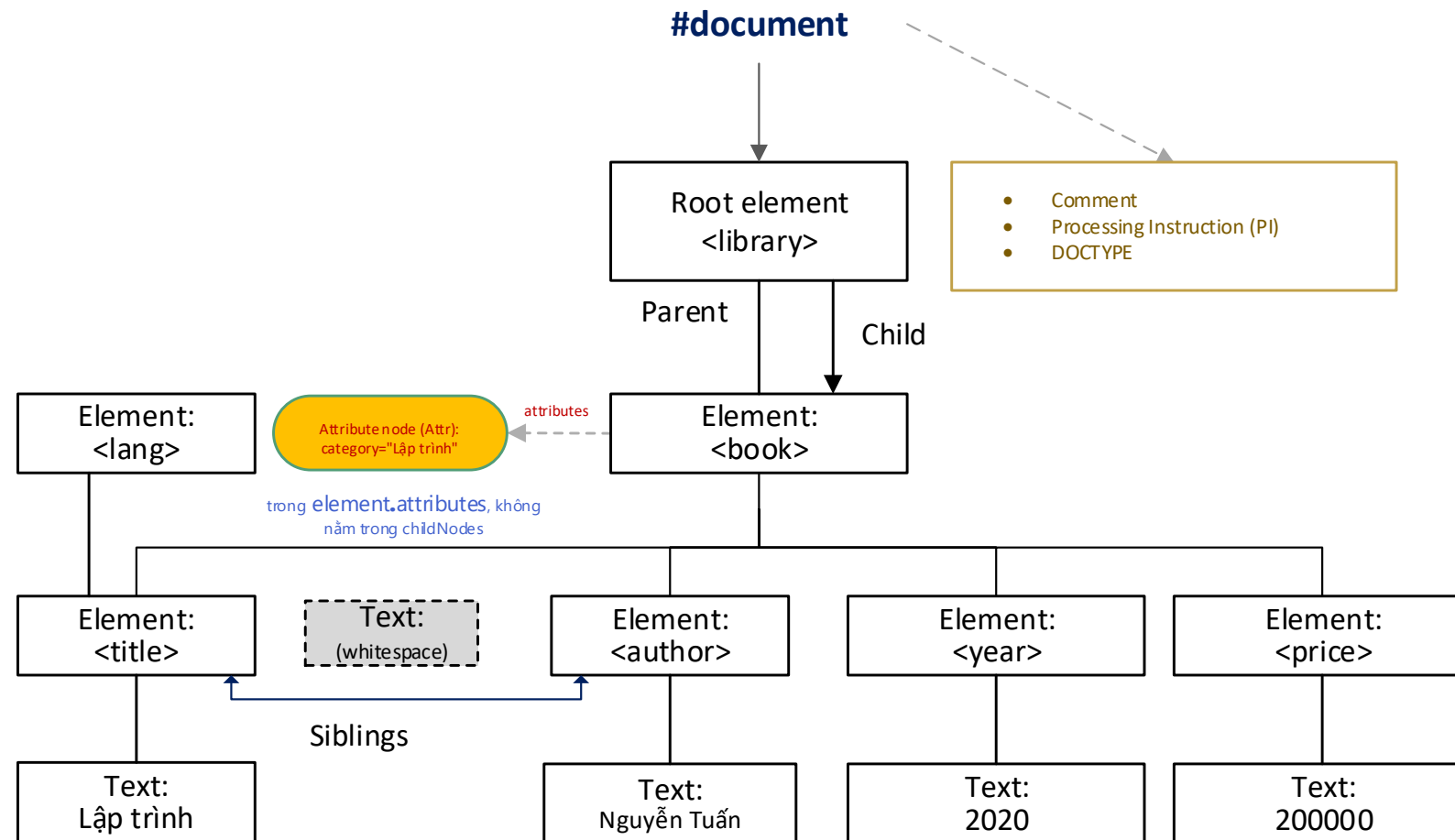
Tất cả các nút khác trong tài liệu được chứa trong <book>. Nút gốc <library> chứa 2 nút <book>. Nút <book> đầu tiên chứa các nút con:

<title>, <author>, <year> và <price>.

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!-- This is a comment node -->
3  <?xml-stylesheet type="text/xsl" href="style.xsl"?>
4  <!DOCTYPE library SYSTEM "library.dtd">
5  <library>
6      <book category="Lập trình">
7          <title lang="vn">Lập trình R</title>
8          <author>Nguyễn Tuấn</author>
9          <year>2020</year>
10         <price currency="VND">200000</price>
11     </book>
12     <book category="web">
13         <title lang="en">Learning XML</title>
14         <author>Erik T.Ray</author>
15         <year>2003</year>
16         <price currency="USD">39.95</price>
17     </book>
18 </library>
```

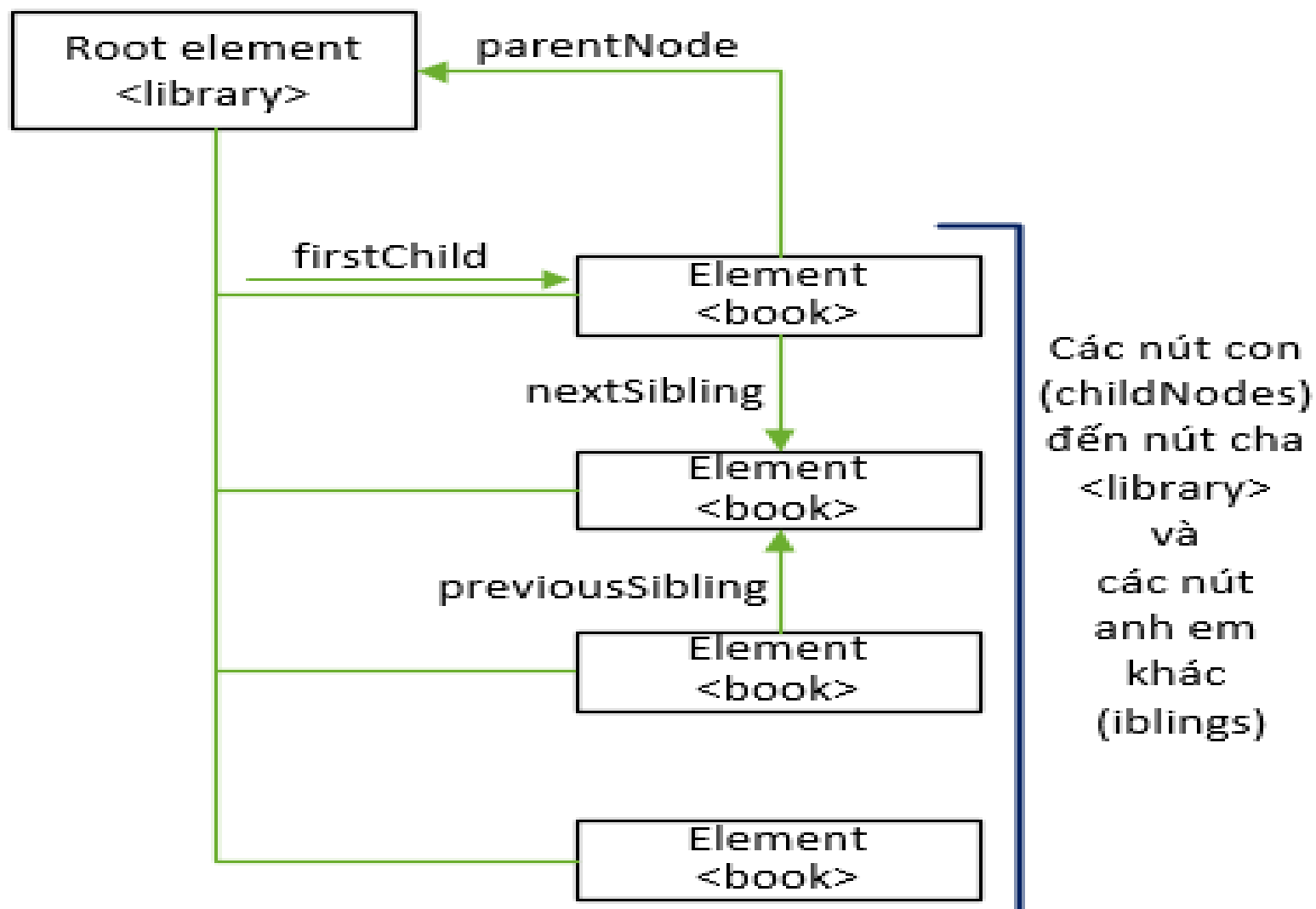
Mỗi nút con chứa một nút văn bản, " Lập trình R ", "Nguyễn Tuấn", "2020", và "200000".

c. Cây XML DOM:



Hình 2.2. Nút cha (Node Parents), con(Children), và Anh/Chị em (Siblings)

- Trong một cây nút, nút trên cùng được gọi là gốc(**root**).
- Mỗi nút, ngoại trừ nút gốc, có đúng một nút cha (**Parent**).
- Một nút có thể có bất kỳ số lượng con(**Children**) nào.
- Một lá (**leaf**) là một nút không có con.
- Các nút anh/chị em(**Sibling**) là các nút có cùng nút cha.



Hình 2.3. Minh họa quan hệ các nút con với nút cha và anh em.

d. Thao tác với DOM

minidom: minidom là một thư viện trong Python để phân tích và xử lý tài liệu XML. Nó cho phép người dùng tạo ra một cây DOM (**D**ocument **O**bject **M**odel) từ một tài liệu XML, sau đó các lập trình viên có thể truy cập và chỉnh sửa các nút trong cây DOM.

Các phương thức chính của minidom bao gồm

- **parse()**: đọc tài liệu XML từ một file hoặc chuỗi, tạo ra một cây DOM và trả về một đối tượng **Document** chứa các nút của cây DOM.
- **getElementsByTagName()**: trả về danh sách các nút có tên nhất định trong cây DOM.
- **createElement()**: tạo ra một nút mới với tên và giá trị được chỉ định.
- **appendChild()**: thêm một nút con vào nút hiện tại.
- **toxml()**: chuyển đổi cây DOM thành một chuỗi XML.

Giả sử chúng ta có tập tin XML, '**employee.xml**' với nội dung như sau:

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <employee>
3      <fname>A</fname>
4      <lname>Nguyễn Văn</lname>
5      <city>Hà Nội</city>
6      <expertise name="SQL"/>
7      <expertise name="Python"/>
8      <expertise name="Testing"/>
9      <expertise name="Business"/>
10 </employee>
```

Sử dụng phương thức 'parse()' của midindom để phân tích tập XML

Ví dụ 2.1.3.1.



```
1 import xml.dom.minidom
2
3 def main():
4     #Sử dụng hàm parse() để đọc và phân tích file "employee.xml sau đó chuyển
5     # đổi nó thành biến đối tượng 'doc'.
6     doc=xml.dom.minidom.parse("employee.xml");
7
8     #In ra In ra tên của node gốc và tag name của node đầu tiên trong file .xml
9     print(doc.nodeName) ← in thuộc tính 'nodeName' của biến đối tượng 'doc'
10    print(doc.firstChild.tagName)
11    if __name__=="__main__": ← Trả về con đầu tiên của node gốc, là "employee"
12        main();
```

Out[: #document
employee



Nếu muốn phần tử gốc (<employee>), dùng **doc.documentElement** thay vì **doc.firstChild**.

Vì **doc.firstChild** phụ thuộc vào thứ tự nút con (thường là Processing Instruction, Comment, hoặc DOCTYPE trước phần tử gốc).

Gọi danh sách các thẻ XML từ tài liệu XML và được in ra



- ❑ Sử dụng phương thức '***getElementsByTagName()***' để lấy danh sách các phần tử từ tập tin XML, sau đó in ra tên của từng phần tử.

Ví dụ 2.1.3.2

```
1  import xml.dom.minidom
2
3  def main():
4      # use the parse() function to load and parse an XML file
5      doc = xml.dom.minidom.parse("employee.xml");
6
7      # print out the document node and the name of the first child tag
8      print(doc.nodeName)
9      print(doc.firstChild.tagName)
10
11     # get a list of XML tags from the document and print each one
12     expertise = doc.getElementsByTagName("expertise")
13     print("%d expertise:" % expertise.length)
14     for skill in expertise:
15         print(skill.getAttribute("name"))
16 if __name__=="__main__":
17     main();
```



#document
employee
4 expertise:
SQL
Python
Testing
Business

Tạo thuộc tính mới bằng cách sử dụng phương thức createElement()

Sử dụng phương thức '**createElement**' hàm createElement() tạo thuộc tính mới, sau đó nối phần thuộc tính thẻ này vào các thẻ XML hiện có.

Ví dụ 2.1.3.3.

Xem xét file trong file '*employee.xml*' chứa thông tin nhân viên với các thuộc tính: **tên, họ, tỉnh/thành phố và kỹ năng**, trong đó thuộc tính kỹ năng hiện có bốn thuộc tính con là: SQL, Python, Testing, Business.

Viết chương trình vidu2.1.3.3.py thực hiện các công việc:

- + Bổ sung thêm một thuộc tính con mới cho thuộc tính kỹ năng hay thêm một thẻ mới 'AI' trong tập tin XML.
- + In thẻ XML

Ví dụ 2.1.3.3. minh họa sử dụng phương thức `createElement()`

```
1  import xml.dom.minidom
2
3  def main():
4      # use the parse() function to load and parse an XML file
5      doc = xml.dom.minidom.parse("employee.xml");
6
7      # print out the document node and the name of the first child tag
8      print(doc.nodeName)
9      print(doc.firstChild.tagName)
10
11     # get a List of XML tags from the document and print each one
12     expertise = doc.getElementsByTagName("expertise")
13     print("%d expertise:" % expertise.length)
14     for skill in expertise:
15         print(skill.getAttribute("name"))
16
17     #Write a new XML tag and add it into the document
18     newexpertise = doc.createElement("expertise")
19     newexpertise.setAttribute("name", "AI")
20     doc.firstChild.appendChild(newexpertise)
21     print(" ")
22
23     expertise = doc.getElementsByTagName("expertise")
24     print("%d expertise:" % expertise.length)
25     for skill in expertise:
26         print(skill.getAttribute("name"))
27
28 if __name__=="__main__":
29     main();
```

Dòng 18-20, tạo và thêm một phần tử 'expertise' mới vào tệp XML.

Dòng 23, lấy danh sách các phần tử 'expertise' hiện có từ tệp XML.

Dòng 25-26, dùng vòng for với mỗi phần tử 'expertise' in ra name từ thuộc tính 'name' của phần tử đó

Kết quả thực hiện chương trình.

#document

employee

4 expertise:

SQL

Python

Testing

Business

5 expertise:

SQL

Python

Testing

Business

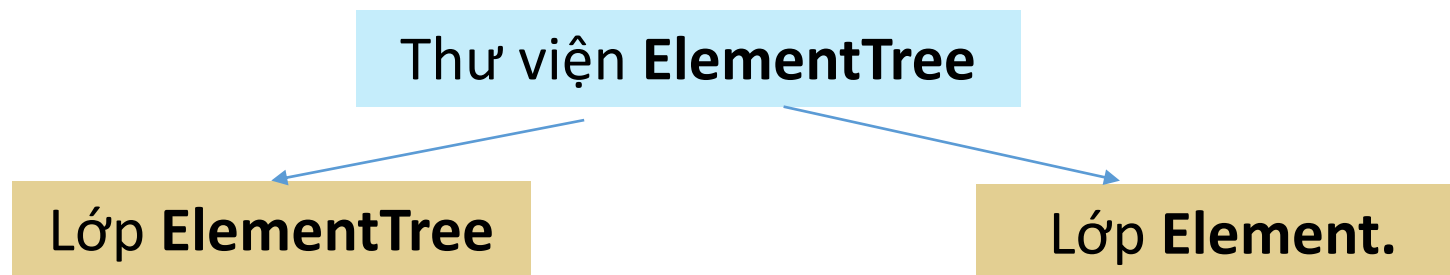
AI

2.1.4. Phân tích tài liệu XML bằng ElementTree

a. *xml.etree.ElementTree*

- **xml.etree.ElementTree** viết tắt là **ET** là một thư viện xử lý XML tích hợp sẵn trong Python,
- **ET** cung cấp một giao diện **API** dễ sử dụng để phân tích và tạo dữ liệu XML.
- Cung cấp các lớp và phương thức để tạo và thao tác các thành phần của tệp XML như các phần tử, thuộc tính, văn bản và các đối tượng khác.
- Hỗ trợ các tính năng quan trọng như phân tích cú pháp XML, truy cập tài liệu theo đường dẫn, tìm kiếm, sửa đổi và thêm các phần tử và thuộc tính.

Trong thư viện ElementTree, có hai lớp đại diện cho cây XML là :
lớp **ElementTree** & lớp **Element**.



- Lớp **ElementTree** được sử dụng để **đại diện cho toàn bộ tài liệu XML** dưới dạng cây. Ví dụ, nó cho phép người dùng đọc và ghi nội dung của tệp XML.
- Lớp **Element** được sử dụng để **đại diện cho một nút duy nhất** trong cây này.
 - Mỗi nút tương ứng với một thẻ trong tài liệu XML và có thể có các thuộc tính và nội dung riêng của nó.
 - Lớp Element cho phép truy cập đến các thuộc tính và nội dung của một nút cụ thể trong cây, và thực hiện các thao tác trên nút đó.

b. Các bước sử dụng thư viện 'ElementTree'

Giả sử có file 'employee.xml' với nội dung như ở slide 31.

Chúng ta sử dụng thư viện 'ElementTree' với các bước như sau:

Bước 1. Import thư viện ElementTree bằng lệnh:

```
import xml.etree.ElementTree as ET
```

Bước 2. Sử dụng phương thức **ET.parse()** để phân tích file XML và tạo ra một đối tượng cây.

Cú pháp: `tree = ET.parse('filename.xml')`

Biến **tree** chứa một đối tượng của lớp ElementTree và được khởi tạo bằng cách phân tích nội dung của tệp XML 'filename.xml'. Sau khi được khởi tạo, ta có thể sử dụng các phương thức của ElementTree để thao tác với tài liệu XML được lưu trữ trong tree.

Bước 3. sử dụng phương thức **find()** hoặc **findall()**.

Cú pháp:

```
root = tree.getroot()  
name = root.find('name')  
print(name.text)
```

Phương thức **find()** sẽ trả về **phần tử đầu tiên** được tìm thấy trong cây, trong khi **findall()** sẽ trả về một **danh sách các phần tử**.

Bước 4. Truy cập các thuộc tính của một phần tử bằng cách sử dụng thuộc tính **attrib** và truy cập các giá trị bằng cách sử dụng tên thuộc tính:

```
expertise = root.find('expertise')  
print(expertise.attrib['name'])
```

Bước 5. Sử dụng thuộc tính **text** của đối tượng Element là 'city' để truy cập và in ra nội dung văn bản của phần tử 'city'

```
city = root.find('city')  
print(city.text)
```

Bước 6. Ta cũng có thể tạo ra một phần tử mới bằng cách sử dụng phương thức **Element()** và thêm nó vào trong cây bằng cách sử dụng phương thức **append()**.

```
newexpertise = ET.Element('expertise')  
newexpertise.attrib['name'] = 'XML'  
root.append(newexpertise)
```

Bước 7. Sử dụng phương thức **write()** để ghi tệp XML đã được cập nhật ra một tệp khác.

```
tree.write('updated.xml')
```


Ví dụ 2.1.4.1. Sử dụng thư viện xml.etree.ElementTree trong Python để phân tích và tạo file XML.



```
1 import xml.etree.ElementTree as ET           #Bước 1
2
3 tree = ET.parse('employee.xml')              #Bước 2
4
5 root = tree.getroot()
6 name = root.find('name')                    #Bước 3
7 print(name.text)
8
9 expertise = root.find('expertise')           #Bước 4
10 print(expertise.attrib['name'])
11
12 city = root.find('city')                    #Bước 5
13 print(city.text)
14
15 newexpertise = ET.Element('expertise')
16 newexpertise.attrib['name'] = 'XML'          #Bước 6
17 root.append(newexpertise)
18
19 tree.write('updated.xml')                    #Bước 7
```



★ **Thư viện XML Minidom**

Tạo toàn bộ cây DOM trong bộ nhớ (Document Object Model) ngay khi tải file XML.

```
import xml.dom.minidom
```

★ **parse()** Đọc tệp XML và Trả về đối tượng Document chứa các nút của DOM tree

- Truy xuất và xử lý dữ liệu XML
- Truy xuất phần tử gốc (root)
- Truy xuất các phần tử con
- Truy xuất nội dung và thuộc tính
- Sửa đổi hoặc thêm dữ liệu

Phù hợp khi cần phân tích chi tiết cấu trúc XML hoặc làm việc với DOM chuẩn.

★ **Thư viện ElementTree**

Dùng cách xử lý dựa trên sự kiện (event-driven) hoặc cây nhẹ hơn, chỉ tải cấu trúc cần thiết vào bộ nhớ

```
import xml.etree.ElementTree as ET
```

★ **parse()** Đọc tệp XML và tạo cây phân tích cú pháp

- Truy xuất và xử lý dữ liệu XML
- Truy xuất phần tử gốc (root)
- Truy xuất các phần tử con
- Truy xuất nội dung và thuộc tính
- Sửa đổi hoặc thêm dữ liệu

Phù hợp khi cần xử lý nhanh hoặc chỉnh sửa file XML



Câu 1: Trong cây DOM, node nào là nút gốc cao nhất của toàn bộ tài liệu XML?

- a) Element Node
- b) Document Node
- c) Text Node
- d) Attribute Node

Câu 2: Trong đoạn XML `<book id="123">Learn XML</book>`, "id" thuộc loại node nào?

- a) Element Node
- b) Attribute Node
- c) Text Node
- d) DocumentType Node

Câu 3: Trong đoạn mã Python `print(doc.documentElement.tagName)`, giá trị in ra là gì nếu file XML bắt đầu bằng `<employee>`?

- a) #document
- b) employee
- c) None
- d) text

Câu 4: Trong file XML `<?xml version="1.0"?> <library>...</library>`, node con đầu tiên của Document Node là gì?

- a) <library>
- b) Processing Instruction Node
- c) Comment Noded)
- d) DocumentType Node

Câu 5: Phương thức nào trong Python với `xml.etree.ElementTree` dùng để lấy tất cả các thẻ `<expertise>` trong file XML?

- a) `find('expertise')`
- b) `getElementsByTagName('expertise')`
- c) `findall('expertise')`
- d) `getAttribute('expertise')`

Câu hỏi thảo luận

1. Anh/Chị hãy trình bày khái niệm XML?
2. Khái niệm DOM?
3. Cú pháp tài liệu XML?
4. API XML là gì?

Bài tập vận dụng

Bài tập 2.1, 2.2, 2.3, 2.4, 2.5 trang 60 TLHT hoặc trên LMS